МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки і комп’ютерних технологій

Звіт

про виконання лабораторної роботи №8

з курсу «Вибрані розділи науки про дані»

«Збереження знань у графовій базі даних для візуалізації енергетичних даних»

Виконав:
студент групи ФЕІм-13с
Шевчук Максим Вікторович
Перевірив:
Парубочий В. О.

Львів 2026

Тема та мета роботи

Тема — збереження знань візуалізації енергетичних даних у графовій базі Neo4j. Мета — перетворити онтологію попередніх робіт на графову структуру, імпортувати поняття й фактичні дані, розробити запити та перевірити ефект індексації.

Завдання

1. Ознайомитися з графовими базами даних і підключитися до Neo4j.
2. Визначити вузли, властивості та зв’язки для онтології.
3. Підготувати JSON/CSV і виконати імпорт через Cypher.
4. Перевірити повноту даних і візуалізувати графові запити.
5. Створити індекси та виміряти продуктивність складних запитів.

1 Предметна область та середовище

Варіант із лабораторної №1: енергетика, програмне забезпечення, візуалізація. Предметна область охоплює вимірювання електроспоживання, їх часову організацію, аналітичні результати й графічні подання. Основою є семантична онтологія лабораторної №6.

Роботу виконано у Neo4j Sandbox через Neo4j Browser та офіційний Python-драйвер. Віддалений сервер — Neo4j Enterprise 2026.04.0, клієнт — Python 3.11.6 і neo4j 5.28.2. Перетворення OWL виконано RDFLib 7.1.4. Усі наведені основні вимірювання отримано у Sandbox.

2 Перетворення онтології на граф

У property graph вузли та зв’язки мають властивості, а мітки визначають прикладний тип вузла. Створено вузли класів, визначення властивостей, індивіди й записи фактичних даних. Правила відображення наведено в табл. 1.

Таблиця 1 — Відображення знань у Neo4j

Джерело	**Подання у графі**
Клас OWL	Вузол OntologyClass з IRI та name
Ієрархія класів	Зв’язок SUBCLASS_OF
Індивід OWL	Вузол OntologyIndividual з предметними мітками
Тип індивіда	Мітка класу та зв’язок TYPE_OF
Об’єктна властивість	Спрямований зв’язок із provenance та IRI
Властивість даних	Число, bool, рядок або localdatetime
Визначення властивості	Вузол OntologyProperty; DOMAIN і RANGE
Година / прогноз ЛР1	Вузол Lab8Observation / Lab8Forecast

Вихідна онтологія містить 37 класів, 31 об’єктну властивість, 34 властивості даних і 26 індивідів. Їх доповнено 34 085 повними годинами та 9 530 прогнозами з першої роботи. Нове навчання моделей не виконувалося.

Висновки HermiT із №6 перенесено як матеріалізовані факти. Neo4j не виконує OWL-виведення автоматично: кардинальності та несумісність класів не стали обмеженнями Cypher. Повний OWL і 993 вихідні RDF-трійки збережено окремо; відновлений RDF-граф ізоморфний джерелу.

3 Підготовка даних та імпорт

Підготовлено nodes.json і relationships.json, а також CSV-подання. Кожен вузол має спільну мітку Lab8Entity та унікальний id. Префікси class:, property:, owl:, data:, measurement: і forecast: розрізняють схему, приклади й повний масив. Час джерела без UTC-зсуву перетворено на localdatetime.

Імпортер передає параметризовані списки пакетами до 1000 записів, створює вузли через MERGE, а потім зв’язує їх за id. Використано обмеження унікальності:

```
CREATE CONSTRAINT lab8_entity_id IF NOT EXISTS
FOR (n:Lab8Entity) REQUIRE n.id IS UNIQUE;
```

Після імпорту отримано 43 746 вузлів і 199 287 зв’язків. Результати контрольних запитів у браузері показано на рис. 1 і 2; вони збігаються з перевіркою імпортера.

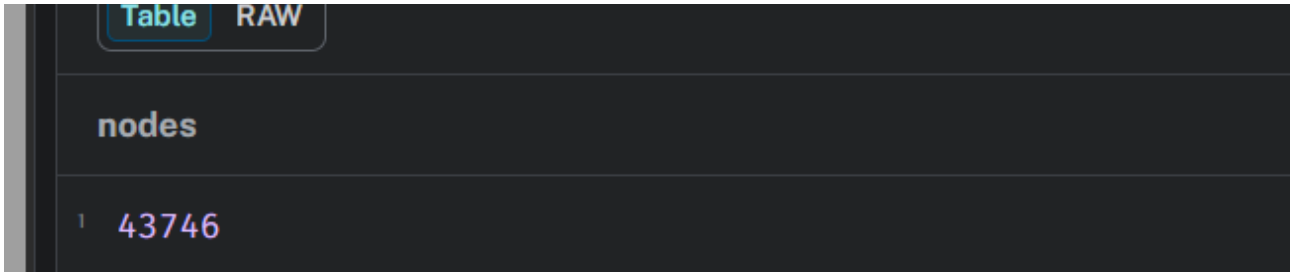


Рис. 1 — Кількість вузлів повного графа

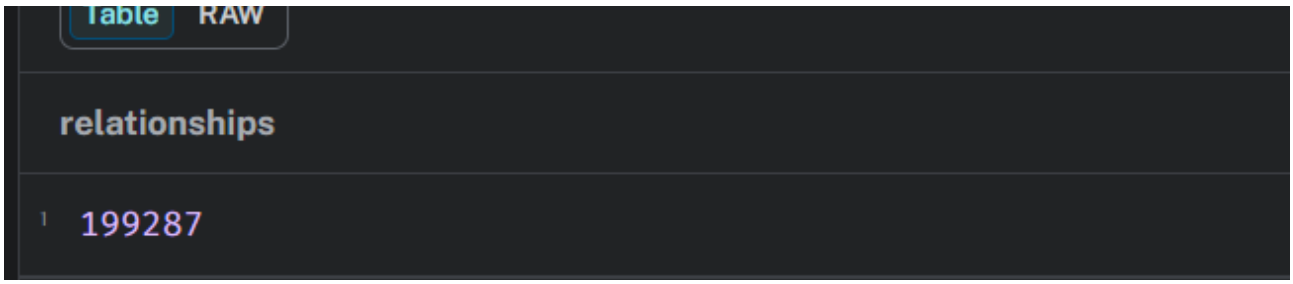


Рис. 2 — Кількість зв’язків повного графа

Окремий локальний повтор імпорту підтвердив ідемпотентність: 0 нових вузлів і 0 нових зв’язків. Цей контроль не підміняє віддалений запуск; перевірка Sandbox має власні відповіді та журнали.

4 Класи та успадкування

Ієрархію подано 37 вузлами OntologyClass та 36 зв’язками SUBCLASS_OF. Запит повертає шляхи від підкласів до батьківських класів; стрілки позначають напрям спеціалізації. Приклад відображення та властивості кореневого DomainEntity наведено на рис. 3.

```
MATCH p=(c:Lab8Entity:OntologyClass)
  -[:SUBCLASS_OF]->
  (parent:Lab8Entity:OntologyClass)
RETURN p;
```

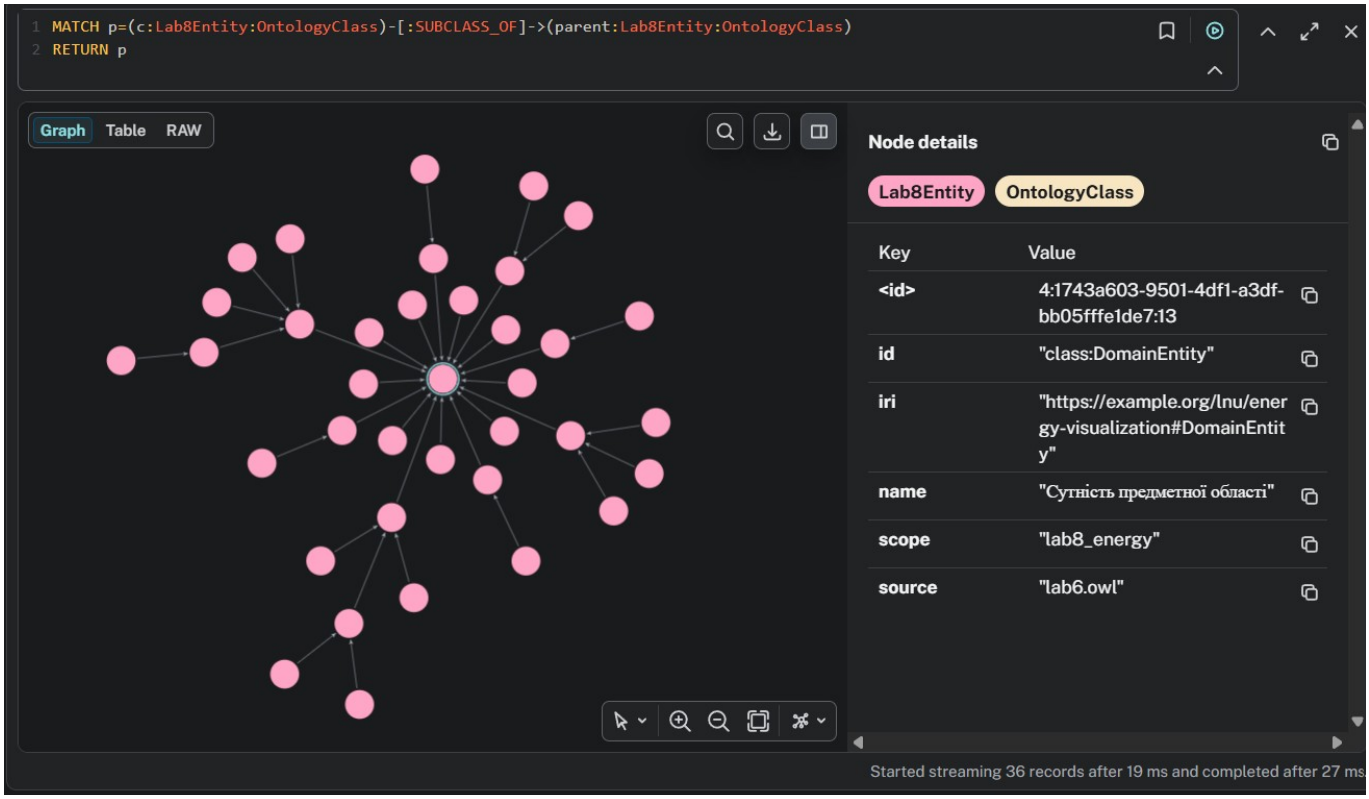


Рис. 3 — Ієрархія класів і властивості DomainEntity

Наприклад, Household спеціалізує ConsumptionObject, EnergyMeasurement — Measurement, а RegressionModel — AnalyticalModel. Мітки й зв’язки TYPE_OF зберігають типи початкових індивідів, включно з відомими висновками та успадкуванням.

5 Граф онтологічних індивідів

Візуалізовано 26 початкових індивідів із предметними відношеннями (рис. 4). До них належать домогосподарство, канали обліку, часові ряди, аналітична модель, прогноз, подання й користувач. Перегляд цього фрагмента дозволяє дослідити структуру без одночасного малювання всіх вимірювань.

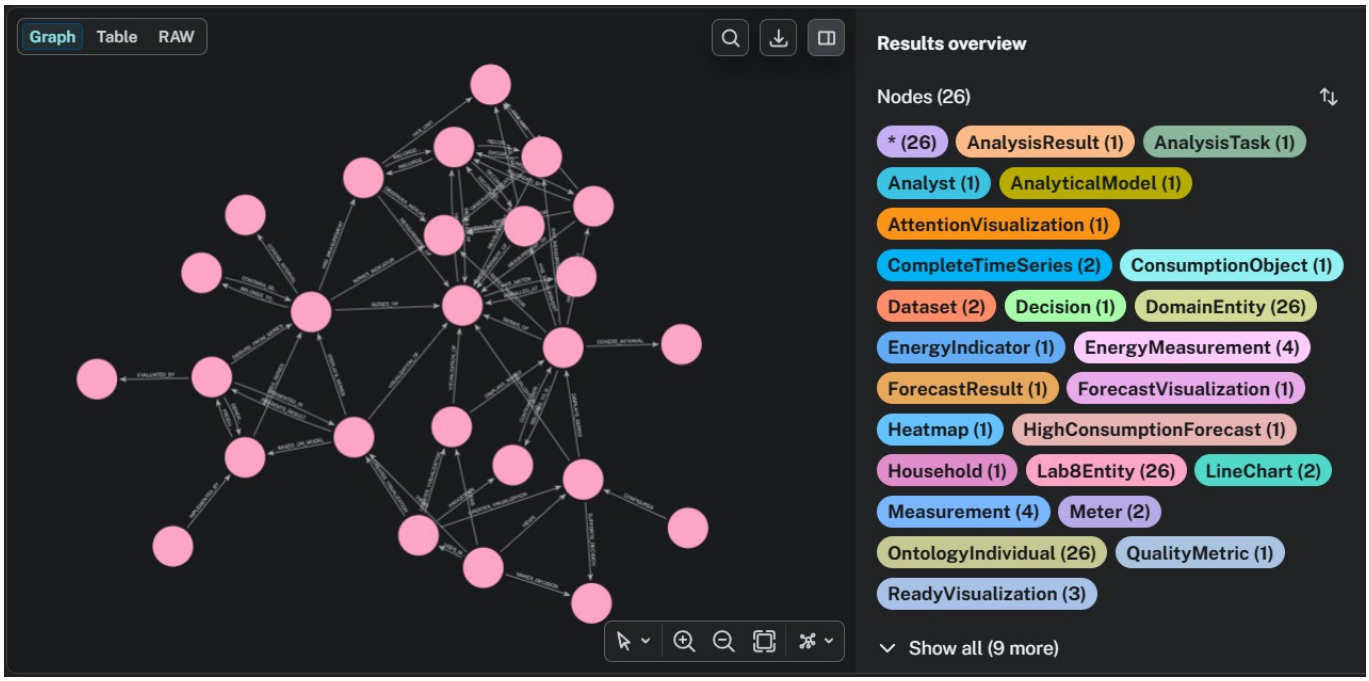


Рис. 4 — Онтологічні індивіди та їхні зв’язки

Зв’язок ANALYZES_SERIES поєднує Random Forest із цільовим тестовим рядом. Він задає походження результату, а не повний перелік лагових ознак моделі першої роботи.

6 Походження прогнозу та семантика

Запит від графіка прогнозу через PRESENTS_RESULT і GENERATED_BY відновлює ланцюжок «подання → результат → модель» (рис. 5). Подання має тип AttentionVisualization, а прогноз — HighConsumptionForecast; ці висновки отримано у №6 й збережено під час імпорту.

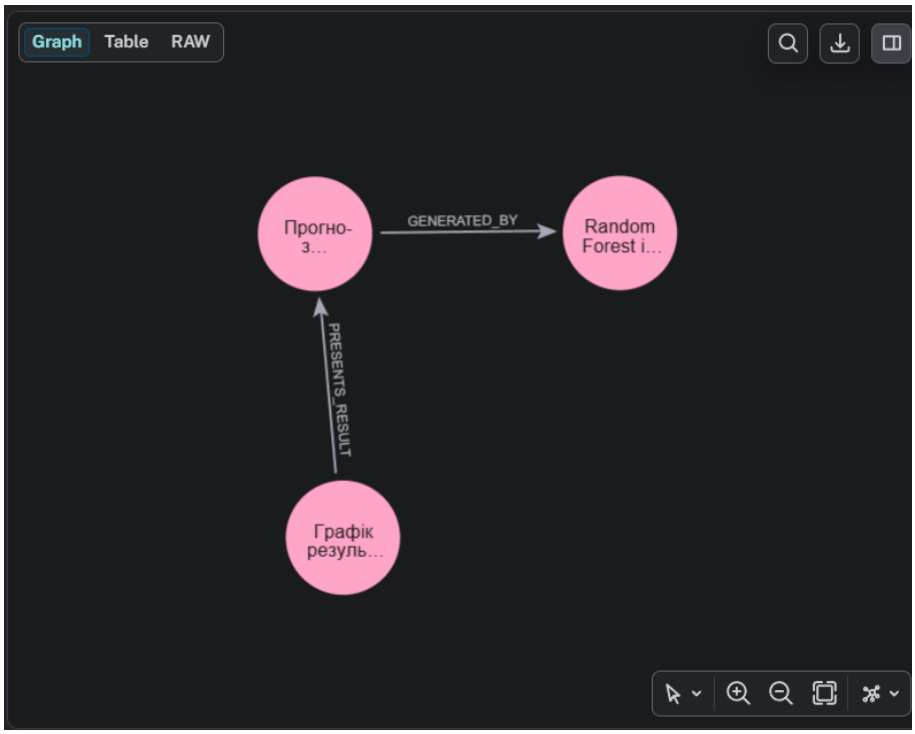


Рис. 5 — Походження прогнозу Random Forest

Значення 1,9519405504171758 kWh перевищує навчальний поріг 1,5613 kWh. Табличний результат із типами показано на рис. 6. Це окреме правило числового прогнозу: мітка Logistic Regression для цього прикладу залишилася 0. Поріг не є технічною нормою безпеки.

forecast_kwh	visualization_types	forecast_types
1.9519405504171758	["Lab8Entity", "DomainEntity", "OntologyIndividual", "Visualization", "AttentionVisualization", "ReadyVisualization", "LineChart", "ForecastVisualization"]	["Lab8Entity", "AnalysisResult", "ForecastResult", "HighConsumptionForecast", "DomainEntity", "OntologyIndividual"]

Рис. 6 — Числовий прогноз і матеріалізовані типи

7 Контекст реального вимірювання

Для вимірювання за 09.09.2009 о 20:00 відновлено чотири контекстні зв’язки: MEASUREMENT_OF, RECORDED_BY, OBSERVES_INDICATOR та HAS_UNIT. На рис. 7 показано вимірювання, домогосподарство, канал загального споживання, показник енергії та одиницю kWh.

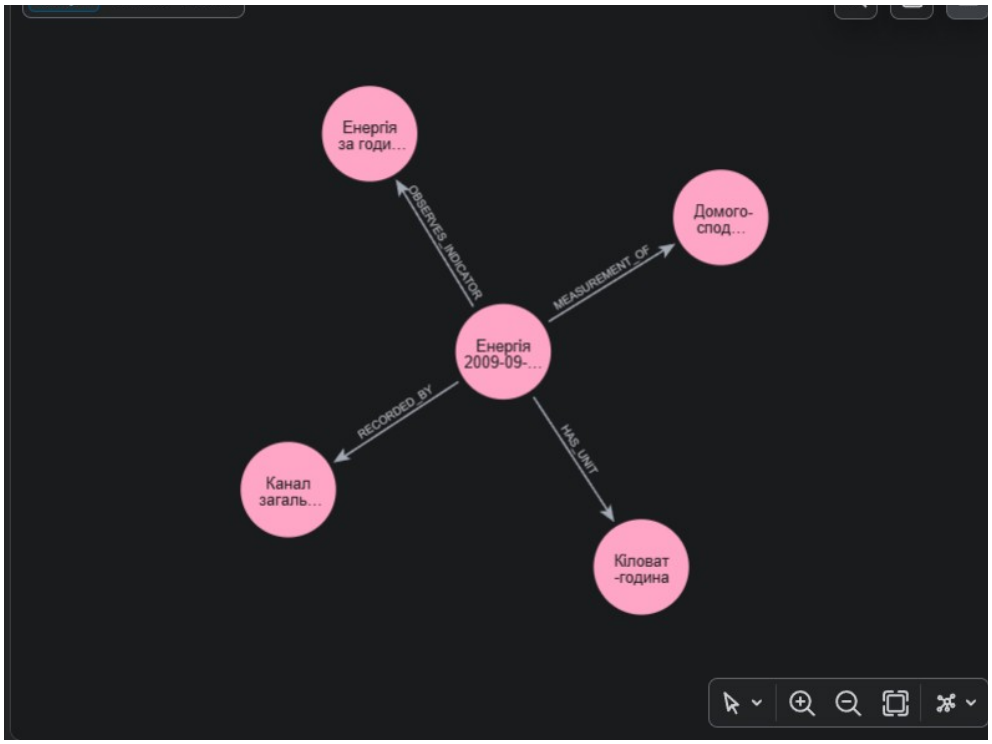


Рис. 7 — Контекст одного годинного вимірювання

```
MATCH (m:Lab8Entity {
  id:'measurement:2009-09-09T20:00:00'})
  -[r:MEASUREMENT_OF|RECORDED_BY|HAS_UNIT|
       OBSERVES_INDICATOR]->(n:Lab8Entity)
RETURN m,r,n;
```

Енергія цього інтервалу становить 2,12 kWh. numericValue зберігає енергію, а activePowerKw — середню активну потужність. Одиниці цих величин розрізняються. Початкові чотири приклади OWL мають окремі id; запити повної вибірки використовують Lab8Observation, тому приклади не подвоюють суму енергії.

8 Запити для отримання знань

Підготовлено й виконано 11 сценаріїв. Часові запити використовують інтервал [start, end); основний приклад — 09.09.2009. Каталог і фактичні відповіді подано у табл. 2. Повні результати збережено в query_results.json.

Таблиця 2 — Запити та результати

№	Призначення	Результат
Q1	Кількість вузлів	43 746
Q2	Ієрархія класів	36 шляхів успадкування
Q3	Домогосподарство → засоби обліку	2 шляхи
Q4	Подання → прогноз → модель	1 шлях, 1,951941 kWh
Q5	Годинні виміри за добу	24 записи
Q6	Прогнози вище 1,5613 kWh	2 записи
Q7	Добова енергія та об’єкт	24 год; 25,355733 kWh
Q8	Похибки добових прогнозів	4 прогнози; MAE 0,429294
Q9	Прогноз → факт → об’єкт	2 високі прогнози
Q10	Контекст одного вимірювання	5 вузлів; 4 зв’язки
Q11	MAE повної тестової вибірки	9 530 прогнозів

Чотири прогнози в добовому прикладі відповідають початку тестового періоду ввечері 09.09.2009. Їх MAE не є оцінкою всієї тестової вибірки. У Q9 максимальна абсолютна похибка серед двох відібраних прогнозів становить 0,430059 kWh.

9 Контроль значень і повноти

Похибку регресії повторно обчислено безпосередньо в Neo4j через зв’язок прогнозу з фактичним вимірюванням. Використано всі 9 530 збережених прогнозів. Відповідний запит і результат показано на рис. 8.



Рис. 8 — MAE на повній тестовій вибірці

Середня абсолютна похибка визначається як $MAE = (1/n)\ \Sigma\ |\hat{y}_i - y_i|$, де n — кількість прогнозів, $\hat{y}_i$ — прогноз, y_i — фактична енергія. Для Random Forest отримано 0,335966 kWh; для базового прогнозу незмінного навантаження — 0,423517 kWh. Результати збігаються з №1 в межах похибки обчислень.

Ця перевірка підтверджує збереження значень та правильність зв’язування прогнозів із фактами, а не нове навчання моделі. Усі цільові мітки часу та actual_kwh перевірено за hourly_clean.csv.

Повний часовий ряд має 502 відсутні години всередині його меж. Наявні 34 085 записів відповідають повним годинам; пропущені інтервали не заповнювалися. Дослідження стосується одного домогосподарства Франції з UCI, а не всієї енергосистеми.

10 Індекси та доступ до даних

Створено унікальне обмеження id та три додаткові range-індекси. Для перевірки стану виконано SHOW INDEXES і db.awaitIndexes(300). Усі чотири індекси lab8_* мають стан ONLINE; їхні призначення наведено в табл. 3.

Таблиця 3 — Індексовані властивості

Мітка та властивості	Призначення
Lab8Entity.id	Унікальність і зв’язування MERGE
Lab8Entity.name	Пошук за назвою
Lab8Observation: objectId, timestamp	Відбір об’єкта й інтервалу часу
Lab8Forecast.targetTime	Відбір прогнозів за часом



Рис. 9 — Перегляд стану індексів у Neo4j Browser

На рис. 9 видно верхню частину таблиці. Повний стан часових індексів підтверджено indexes.json із того самого запуску. Композитний індекс прискорює початковий пошук за objectId і timestamp; індекс targetTime — пошук потрібного діапазону прогнозів.

11 Тестування продуктивності

На одному повному графі порівняно Q7, Q8 та Q9 до й після додавання пошукових індексів. Обмеження id існувало в обох умовах. Кожен сценарій прогрівався тричі; далі виконано по 10 повторів із чергуванням порядку запитів. PROFILE запускався окремо. Відповіді до й після індексації еквівалентні.

Таблиця 4 — Результати вимірювань у Sandbox

Запит	Медіана до, ms	Медіана після, ms	DB Hits до	DB Hits після
Q7	205,719	126,806	102400	217
Q8	141,237	132,952	19109	53
Q9	123,897	124,034	19087	31

Для Q7 медіана зменшилася на 38,4%, для Q8 — на 5,9%. Для Q9 зміна з 123,897 до 124,034 ms означає практично однаковий час; прискорення не виявлено. DB Hits скоротилися в усіх трьох сценаріях.

Таблиця 5 — Повернені дані та варіація часу

Запит	Рядків	JSON, байт	P95 до, ms	P95 після, ms
Q7	1	123	313,723	218,444
Q8	1	120	252,301	199,327
Q9	1	114	221,522	222,715

wall_ms охоплює виконання, мережевий обмін, отримання й серіалізацію результату. Значна варіація повторів обмежує висновки про невеликий виграш. JSON-байти — обсяг серіалізованої відповіді, а DB Hits — звернення рушія; це не точні мережеві байти або дисковий I/O.

12 Аналіз планів виконання

Збережені PROFILE-плани показують зміну початкового доступу з NodeByLabelScan на NodeIndexSeek або NodeIndexSeekByRange. У табл. 6 наведено фактичні оператори Q7 з before_profiles.json та after_profiles.json. Це подання виміряного плану, а не скриншот інтерфейсу.

Таблиця 6 — Оператори плану Q7

Умова	Оператор	Rows	DB Hits
До	ProduceResults	1	0
До	Sort	1	0
До	EagerAggregation	1	72
До	Filter	24	48
До	Expand(All)	24	24
До	Filter	24	68170
До	NodeByLabelScan	34085	34086
Після	ProduceResults	1	0
Після	Sort	1	0
Після	EagerAggregation	1	96
Після	Filter	24	48
Після	Expand(All)	24	48
Після	NodeIndexSeek	24	25

До індексації скануються 34 085 вузлів вимірювань. Після індексації початковий пошук повертає 24 потрібні години. Сума DB Hits плану зменшилася зі 102 400 до 217. Для Q8 і Q9 використано NodeIndexSeekByRange за targetTime.

Ранні часові фільтри зменшують подальшу роботу переходів графом. Повернення одного агрегованого рядка в усіх умовах дозволяє порівнювати однакові результати; менша кількість DB Hits не гарантує пропорційного зменшення загального клієнтського часу.

13 Перевірка реалізації

Віддалений запуск завершився статусом passed. Перевірено кількість вузлів і кожного типу зв’язків, збіг усіх прогнозів із фактами, добову суму енергії, MAE, семантичні типи та рівність відповідей до/після індексів. JSON/CSV і джерела мають контрольні суми; збережено всі повтори та сирі плани.

Повні результати містяться в results/sandbox_20261016T054342525797Z. Оригінали 13 зображень Neo4j Browser збережено без редагування. Для звіту використано репрезентативні фрагменти; непоказані в кадрі деталі підтверджуються структурованими файлами.

Висновок

Перетворено семантичну онтологію енергетичних даних на граф Neo4j і виконано повний імпорт 43 746 вузлів та 199 287 зв’язків. Запити відновлюють ієрархію класів, контекст вимірювання, походження прогнозу й відомі семантичні типи. Повторний розрахунок MAE на 9 530 прогнозах підтвердив збереження результатів першої роботи.

Індекси істотно зменшили обсяг роботи рушія. Для добової агрегації медіана часу зменшилася на 38,4%; для пошуку шляхів прогнозів прискорення за часом не виявлено. Отже, ефект оптимізації слід оцінювати за планами, DB Hits і повторними вимірюваннями, враховуючи мережеві витрати. Повна OWL-семантика залишається у вихідній онтології; у Neo4j зберігається її прикладна проєкція.

Використані джерела

1. Умова лабораторної роботи №8 «Збереження знань у граф базі даних».

2. UCI Individual Household Electric Power Consumption. DOI: 10.24432/C58K54. Ліцензія CC BY 4.0.

3. Документація Neo4j: CREATE INDEX, Query tuning та Understanding query plans; neo4j.com/docs/cypher-manual/current/.

4. Онтологія й перевірені висновки лабораторної №6; збережені результати лабораторної №1.

Додаток А Основні функції імпорту

build_lab8.py та run_neo4j.py — фрагменти реалізації

```
def node_statement(labels, temporal):
    query = 'UNWIND $rows AS row\nMERGE (n:Lab8Entity {id: row.id})\nSET n +=
row.properties\n'
    extra = [x for x in sorted(labels) if x != BASE_LABEL]
    if extra:
        query += 'SET n' + ''.join(':' + x for x in extra) + '\n'
    for key in sorted(temporal):
        query += f'SET n.{key} = localdatetime(row.properties.{key})\n'
    return query + 'RETURN count(*) AS processed'

def edge_statement(kind):
    assert re.fullmatch(r'[A-Z][A-Z_0-9]*', kind)
    return ('UNWIND $rows AS row\nMATCH (a:Lab8Entity {id: row.source})\n'
            'MATCH (b:Lab8Entity {id: row.target})\n'
            f'MERGE (a)-[r:{kind}]->(b)\nSET r += row.properties\nRETURN
count(*) AS processed')

def import_graph(session,nodes,edges):
    created_nodes,created_edges=0,0
    groups = list(group_nodes(nodes).items())
    for index,((labels,temporal),rows) in enumerate(groups):
        for offset in range(0,len(rows),1000):
            chunk=rows[offset:offset+1000]
            result=session.run(node_statement(labels,temporal),rows=chunk)
            processed=result.single()['processed']
            summary=result.consume()
            if processed!=len(chunk):
                raise RuntimeError('Node import did not process all rows')
            created_nodes+=summary.counters.nodes_created
        if len(rows)>1000:
            print(f'  Nodes: {index+1}/{len(groups)}, batch group size
{len(rows)}',flush=True)
    for kind,rows in group_edges(edges).items():
        for offset in range(0,len(rows),1000):
            chunk=rows[offset:offset+1000]
```

Додаток А Основні функції імпорту — продовження

Продовження лістингу

```
            result=session.run(edge_statement(kind),rows=chunk)
            processed=result.single()['processed']
            summary=result.consume()
            if processed!=len(chunk):
                raise RuntimeError('Missing endpoint during import: '+kind)
            created_edges+=summary.counters.relationships_created
    return
{'nodes_created':created_nodes,'relationships_created':created_edges}
```

Додаток Б Вимірювання продуктивності

run_neo4j.py — отримання відповідей, PROFILE та повторні вимірювання

```
def run(session, query, parameters=None):
    started = time.perf_counter()
    result = session.run(query, parameters or {})
    rows = [serial(dict(record)) for record in result]
    summary = result.consume()
    elapsed_ms = (time.perf_counter()-started)*1000
    return rows, summary, elapsed_ms

def plan_totals(plan):
    hits, rows, operators = 0, 0, []
    pending = [plan]
    while pending:
        item = pending.pop()
        if not item:
            continue
        hits += item.get('dbHits', 0)
        rows += item.get('rows', 0)
        operators.append(item.get('operatorType', ''))
        pending.extend(item.get('children', []))
    return {'db_hits_sum': hits, 'operator_rows_sum': rows, 'operators':
operators}

def measure(session, specs, params, folder, stage, repeats):
    benchmarks = [q for q in specs if q['benchmark']]
    samples, answers, profiles = [], {}, {}
    for spec in benchmarks:
        for _ in range(3):
            run(session,spec['query'],params)
    for iteration in range(repeats):
        order = benchmarks if iteration % 2 == 0 else
list(reversed(benchmarks))
        for spec in order:
            rows,summary,wall = run(session,spec['query'],params)
            answers[spec['id']] = rows

samples.append({'stage':stage,'query':spec['id'],'iteration':iteration+1,

'wall_ms':wall,'result_available_ms':summary.result_available_after,

'result_consumed_ms':summary.result_consumed_after,
```

Додаток Б Вимірювання продуктивності — продовження

Продовження лістингу

```
                            'returned_rows':len(rows),

'json_utf8_bytes':len(json.dumps(rows,ensure_ascii=False).encode('utf-8'))})
    stats = {}
    for spec in benchmarks:
        rows,summary,_ = run(session,'PROFILE\n'+spec['query'],params)
        plan = serial(summary.profile)
        profiles[spec['id']] = {'plan':plan,**plan_totals(plan)}
        selected = [r for r in samples if r['query']==spec['id']]
        walls = sorted(r['wall_ms'] for r in selected)
        stats[spec['id']] = {'median_wall_ms':statistics.median(walls),

'p95_wall_ms':walls[math.ceil(.95*len(walls))-1],

'min_wall_ms':min(walls),'max_wall_ms':max(walls),
                             'returned_rows':selected[0]['returned_rows'],
                             'json_utf8_bytes':selected[0]
['json_utf8_bytes'],
                             **plan_totals(plan)}
    dump(folder/(stage+'_profiles.json'),profiles)
    dump(folder/(stage+'_answers.json'),answers)
    dump(folder/(stage+'_summary.json'),stats)
    with
(folder/(stage+'_samples.csv')).open('w',encoding='utf-8',newline='') as
handle:
        writer=csv.DictWriter(handle,fieldnames=list(samples[0]))
        writer.writeheader()
        writer.writerows(samples)
    return stats,answers
```

Додаток В Запити Cypher

Обмеження, індекси й основні запити; параметри start, end, objectId та threshold задає клієнт

```
CREATE CONSTRAINT lab8_entity_id IF NOT EXISTS FOR (n:Lab8Entity) REQUIRE
n.id IS UNIQUE;

CREATE RANGE INDEX lab8_observation_time IF NOT EXISTS FOR
(n:Lab8Observation) ON (n.objectId, n.timestamp);
CREATE RANGE INDEX lab8_forecast_time IF NOT EXISTS FOR (n:Lab8Forecast) ON
(n.targetTime);
CREATE RANGE INDEX lab8_entity_name IF NOT EXISTS FOR (n:Lab8Entity) ON
(n.name);
CALL db.awaitIndexes(300);
SHOW INDEXES;

// 02_ontology_graph.cypher
MATCH p=(c:Lab8Entity:OntologyClass)
  -[:SUBCLASS_OF]->
  (parent:Lab8Entity:OntologyClass)
RETURN p;

// 04_semantic_forecast.cypher
MATCH p=(v:Lab8Entity {id:'owl:forecastChart'})
  -[:PRESENTS_RESULT]->
  (f:Lab8Entity)
  -[:GENERATED_BY]->
  (m:Lab8Entity)
RETURN p, f.forecastKwh AS forecast_kwh, labels(v) AS visualization_types,
labels(f) AS forecast_types;

// 07_daily_energy.cypher
MATCH (m:Lab8Observation)
  -[:MEASUREMENT_OF]->
  (h:Lab8Entity)
WHERE m.objectId=$objectId
AND m.timestamp >= localdatetime($start)
AND m.timestamp < localdatetime($end)
RETURN h.name AS household, date(m.timestamp) AS day, count(m) AS hours,
sum(m.numericValue) AS energy_kwh
ORDER BY day;
```

Додаток В Запити Cypher — продовження

Продовження лістингу

```
// 08_forecast_errors.cypher
MATCH (f:Lab8Forecast)
  -[:GENERATED_BY]->
  (model:Lab8Entity)
MATCH (f)
  -[:COMPARED_WITH]->
  (m:Lab8Observation)
WHERE f.targetTime >= localdatetime($start)
AND f.targetTime < localdatetime($end)
RETURN model.algorithm AS algorithm, count(f) AS forecasts,
avg(abs(f.forecastKwh-m.numericValue)) AS mae_kwh, avg(abs(f.persistenceKwh-
m.numericValue)) AS persistence_mae_kwh;

// 09_forecast_paths.cypher
MATCH (f:Lab8Forecast)
  -[:COMPARED_WITH]->
  (m:Lab8Observation)
  -[:MEASUREMENT_OF]->
  (h:Lab8Entity)
WHERE f.targetTime >= localdatetime($start)
AND f.targetTime < localdatetime($end)
AND f.forecastKwh > $threshold
RETURN h.name AS household, count(f) AS high_forecasts,
max(abs(f.forecastKwh-m.numericValue)) AS max_error_kwh;

// 11_all_test_mae.cypher
MATCH (f:Lab8Forecast)
  -[:COMPARED_WITH]->
  (m:Lab8Observation)
RETURN count(f) AS test_rows, avg(abs(f.forecastKwh-m.numericValue)) AS
mae_kwh, avg(abs(f.persistenceKwh-m.numericValue)) AS persistence_mae_kwh;
```